

05	Broken Access Control, Brute Force Attack, IDOR
취약점 개요	
점검 내용	클라이언트 측 정보 노출 점검 - 개발자 도구를 활용하여 JavaScript 파일 분석 - main.js 파일 내 관리자 관련 경로(Path) 탐색 - 숨겨진 관리자 엔드포인트 접근 가능 여부 확인 인증 취약점 점검(Brute Force) - 로그인 요청 패턴 분석 - 특정 사용자 계정(jim)에 대한 반복 로그인 시도 - 로그인 실패 횟수 제한 여부 확인 - 계정 잠금 정책 적용 여부 확인 권한 검증 취약점 점검(IDOR) - Burp Suite Proxy를 통해 HTTP 요청 가로채기 - 주문 조회 요청 파라미터(userId 등) 분석 - 사용자 식별자 값을 변경하여 요청 재전송 - 타 사용자 주문 정보 열람 가능 여부 확인
점검 목적	본 점검의 목적은 웹 애플리케이션 내 인증(Authentication) 및 인가(Authorization) 절차의 적절성 여부를 확인하고, 클라이언트 측 정보 노출 및 서버 측 권한 검증 미흡으로 인한 보안 취약점을 식별함
보안 위협	- OWASP Top 10 에서의 다음 항목에 해당한다. - A01: Broken Access Control - A07: Identification and Authentication Failures - A05: Security Misconfiguration - 점검을 통해 확인된 취약점은 다음과 같은 보안 위협을 초래 할 수 있다. 관리자 기능 무단 접근, 사용자 계정 탈취, 개인정보 및 거래 정보 유출, 법적 책임 발생 가능성이 발생함
참고	Web Frame 특징 Angular Inspector 탭 HTML: <app-root> </app-root> , ng-version="17.0.0" Console: window.ng Vue HTML: data-v-xxxxxx, <div id="app"> Console: window.vue, __VUE_DEVTOOLS_GLOBAL_HOOK__ React Javascript: window.__REACT_DEVTOOLS_GLOBAL_HOOK__ Network(Header 부분): X-Powered-By: Express(css), Server: Node.js(arduino) 에러 페이지 문구: TypeError: Cannot read property at app.js:23:15

HTTP/HTTPS 프로토콜 구조

HTTP(HyperText Transfer Protocol)

클라이언트(브라우저) 와 서버 간 데이터를 주고받는 요청-응답(Request-Response) 기반 프로토콜

구조

- ① Client → HTTP Request → Server
- ② Client ← HTTP Response ← Server

HTTP Request 구조

- ① Request Line
 - Method (GET, POST 등)
 - 요청 경로 (URL)
 - HTTP 버전
- ② Header
 - 요청에 대한 추가 정보
 - 인증 정보, 쿠키, 콘텐츠 타입 등 포함
- ③ Body (POST에서 사용)
 - 서버로 전달할 실제 데이터

HTTP Response 구조

- ① Status Line
- ② Header
- ③ Body

IDOR (Insecure Direct Object Reference)는 사용자가 직접 참조할 수 있는 객체(ID, 파일, 계정 등)를 적절한 권한 검증 없이 접근할 수 있는 취약점이다

Burp Suite

Burp Suite는 웹 애플리케이션의 보안 취약점을 분석하기 위한

웹 프록시 기반 모의해킹(웹 취약점 진단) 도구 Burp Suite가 중간에서 가로채려면
CA 인증서 설치가 필요한 이유가 바로 TLS

기능

1. Proxy

◆ 기능

HTTP/HTTPS 요청 가로채기

요청/응답 수정

패킷 재전송

◆ Example

ID 값 변조 → IDOR 확인

권한 우회 테스트

2. Repeater

◆ 기능

특정 요청을 반복적으로 수정·전송

서버 응답 비교 분석

◆ Example

파라미터 값 하나씩 변경

SQL Injection 테스트

3. Intruder

◆ 기능

자동화된 공격 수행

Brute Force

Payload 삽입

◆ Example

비밀번호 무작위 대입 공격

파라미터 퍼징(Fuzzing)

토큰 예측 테스트

4. Scanner

◆ 기능

자동 취약점 스캔

SQLi, XSS, CSRF 등 자동 탐지

5. Decoder

◆ 기능

인코딩/디코딩

Base64, URL Encoding, Hash 변환

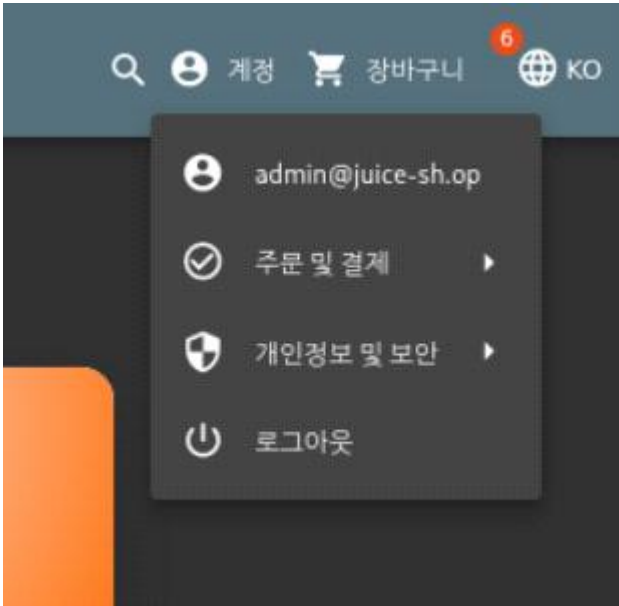
점검대상 및 판단기준

대상	웹 애플리케이션 프론트엔드(HTML, JavaScript), 로그인 인증 가능, 서버 측 권한 검증로직
조치방법	관리자 경로 보호를 위해 관리자 페이지는 서버 측 Role 기반 접근 통제 적용하고 프론트엔드 파일 내 민감 경로 제거함. BruteForce대응 로그인 실패 횟수 제한, 계정 잠금 정책 적용, CAPTCHA 적용해야함. IDOR 취약점 대응 요청 파라미터 기반 데이터 조회 금지, 서버 측에서 세션 사용자와 요청 데이터 일치 여부 검증함.

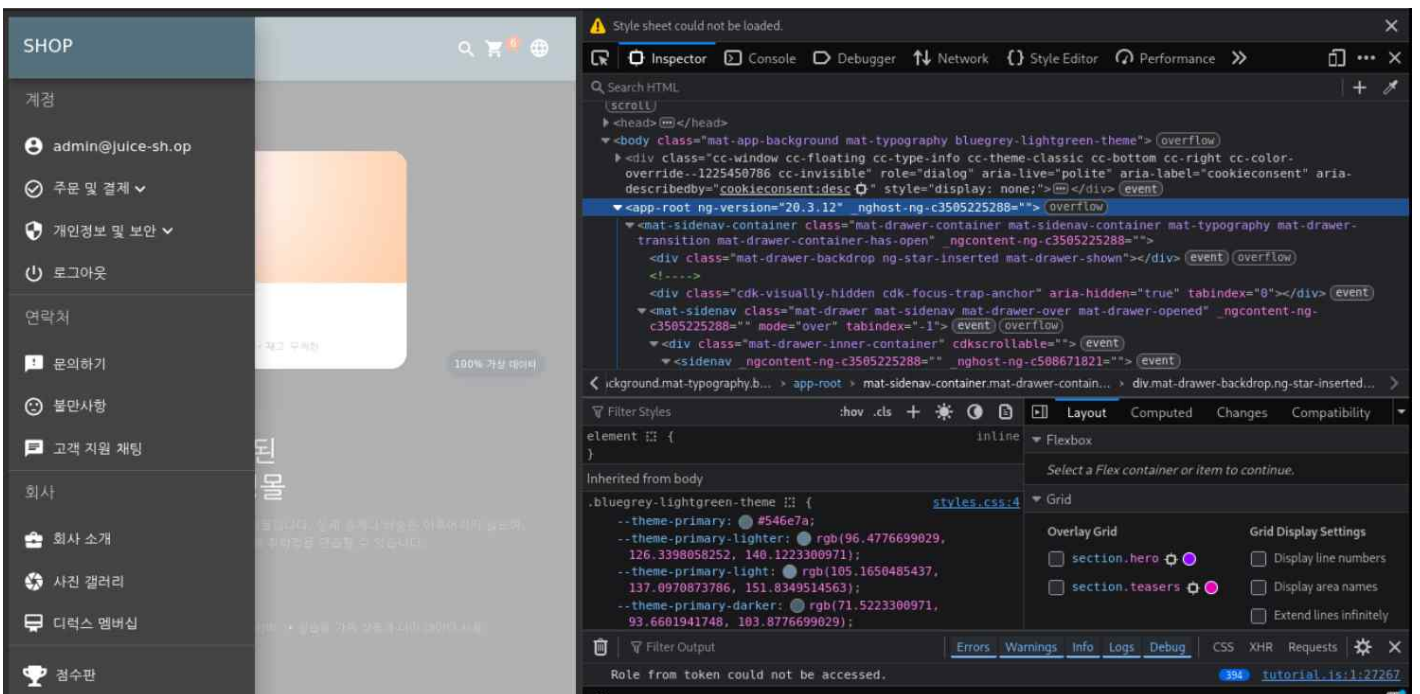
점검 및 조치사례

※Broken Access Control※

1. Injection 공격으로 Admin계정을 탈취한 상태이다.

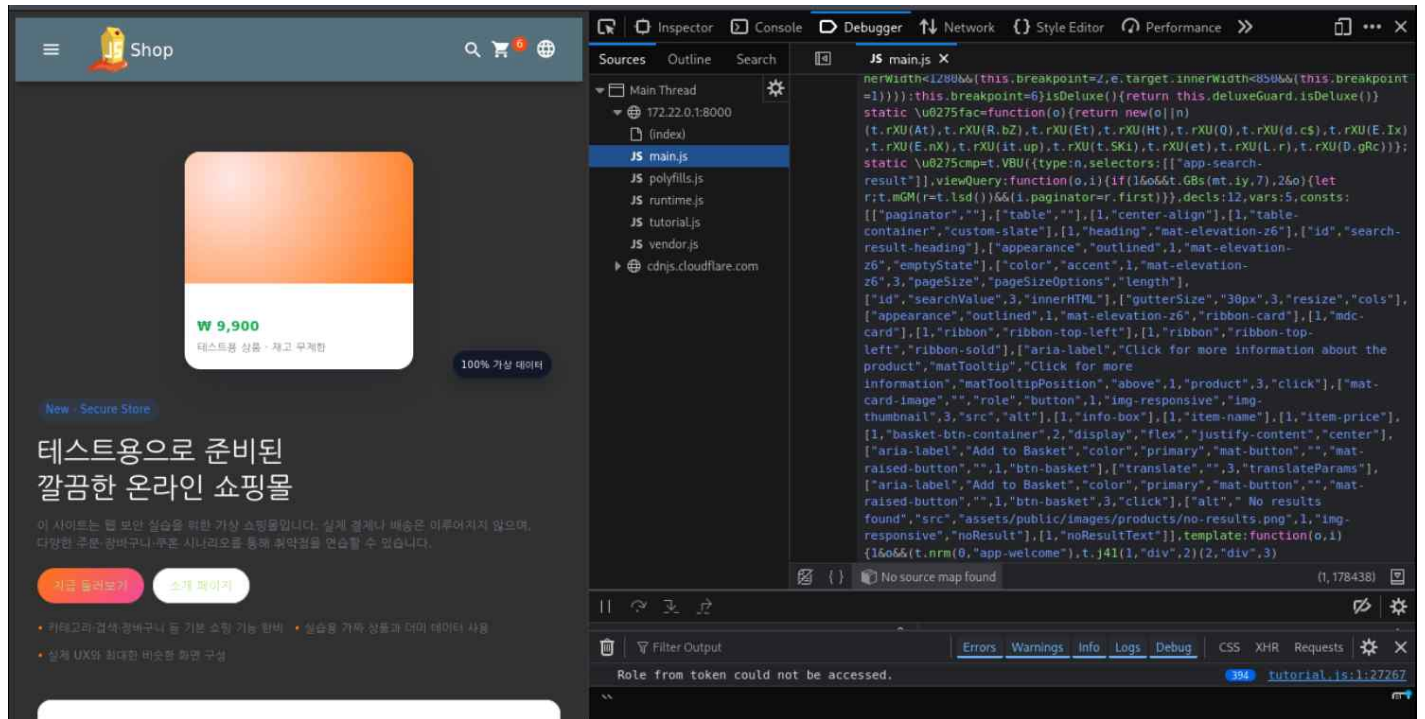


2. F12를 통해 웹페이지는 뷰, 리액트, 앵귤러 중 어떤 구조 확인



<app-root ng-version="20.3.12 _ngghost-ng-c3505225288=">를 통해서 앵귤러의 특징을 알수있음

3. main.js의 코드 구조를 파악 및 관리자 페이지로 예상 되는 페이지를 탐색



“main.js”

생략

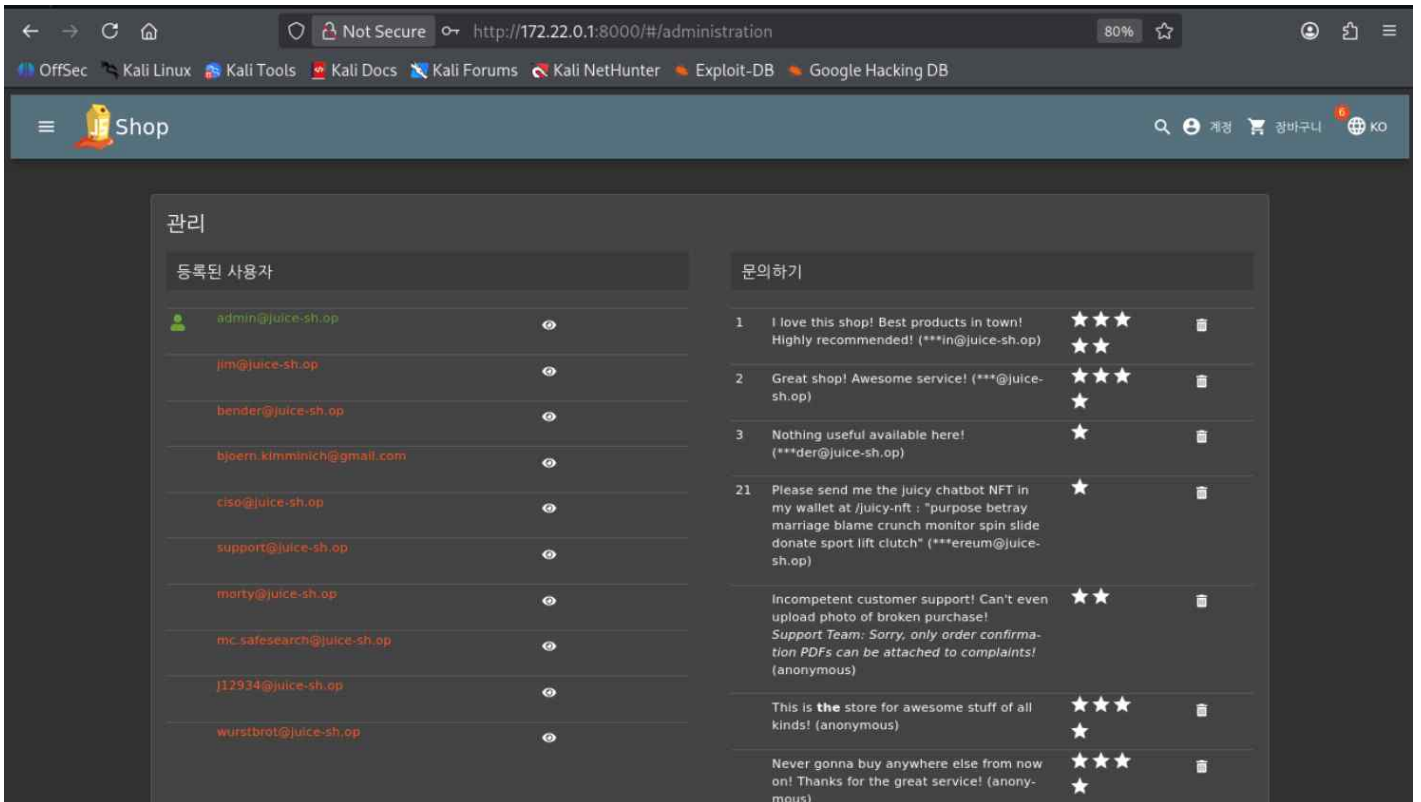
```
Promise.all([m.e(989),m.e(380)]).then(m.bind(m,7380)).Web3SandboxModule});return function(){return n.apply(this,arguments)}}(),
```

```
tp={path:"administration",component:cr,canActivate:[ie]},
{path:"accounting",component:$l,canActivate:[ae]},
{path:"about",component:Ci},
{path:"address/select",component:Es,canActivate:[X]},
{path:"address/saved",component:Rs,canActivate:[X]},
{path:"address/create",component:Re,canActivate:[X]},
{path:"address/edit/:addressId",component:Re,canActivate:[X]},
{path:"delivery-method",component:Gc},
{path:"deluxe-membership",component:ad,canActivate:[X]},
{path:"saved-payment-methods",component:pl},{path:"basket",component:Do},
{path:"order-completion/:id",component:ic},
{path:"contact",component:ri},{path:"photo-wall",component:Zc},
{path:"complain",component:xr},
```

생략

path: administration 이라는 페이지가 있다는 것을 찾았다.

4. http://172.22.0.1:8000/#/administration 접속



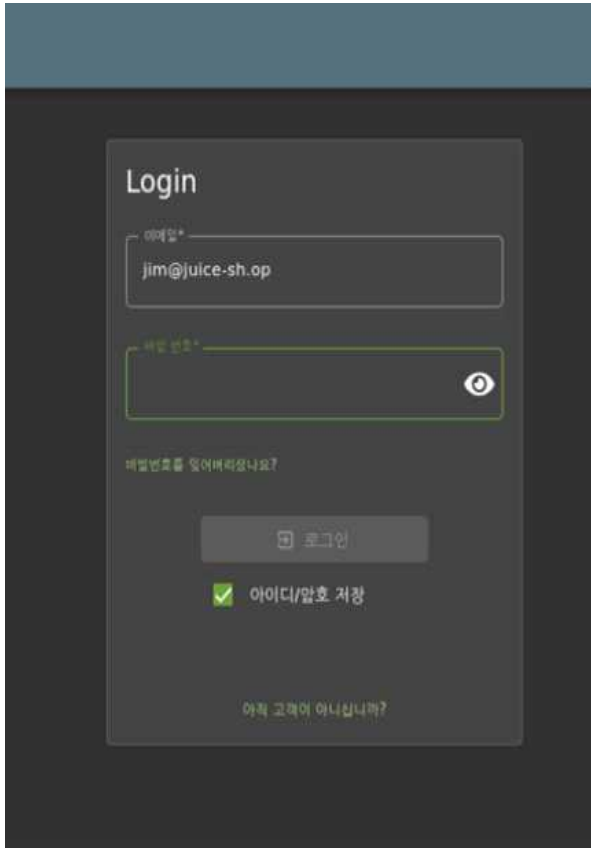
등록된 사용자의 E-MAIL 정보들을 파악하고 문의 내용 또한 확인가능한 관리자 페이지를 찾아냄

ID 정보

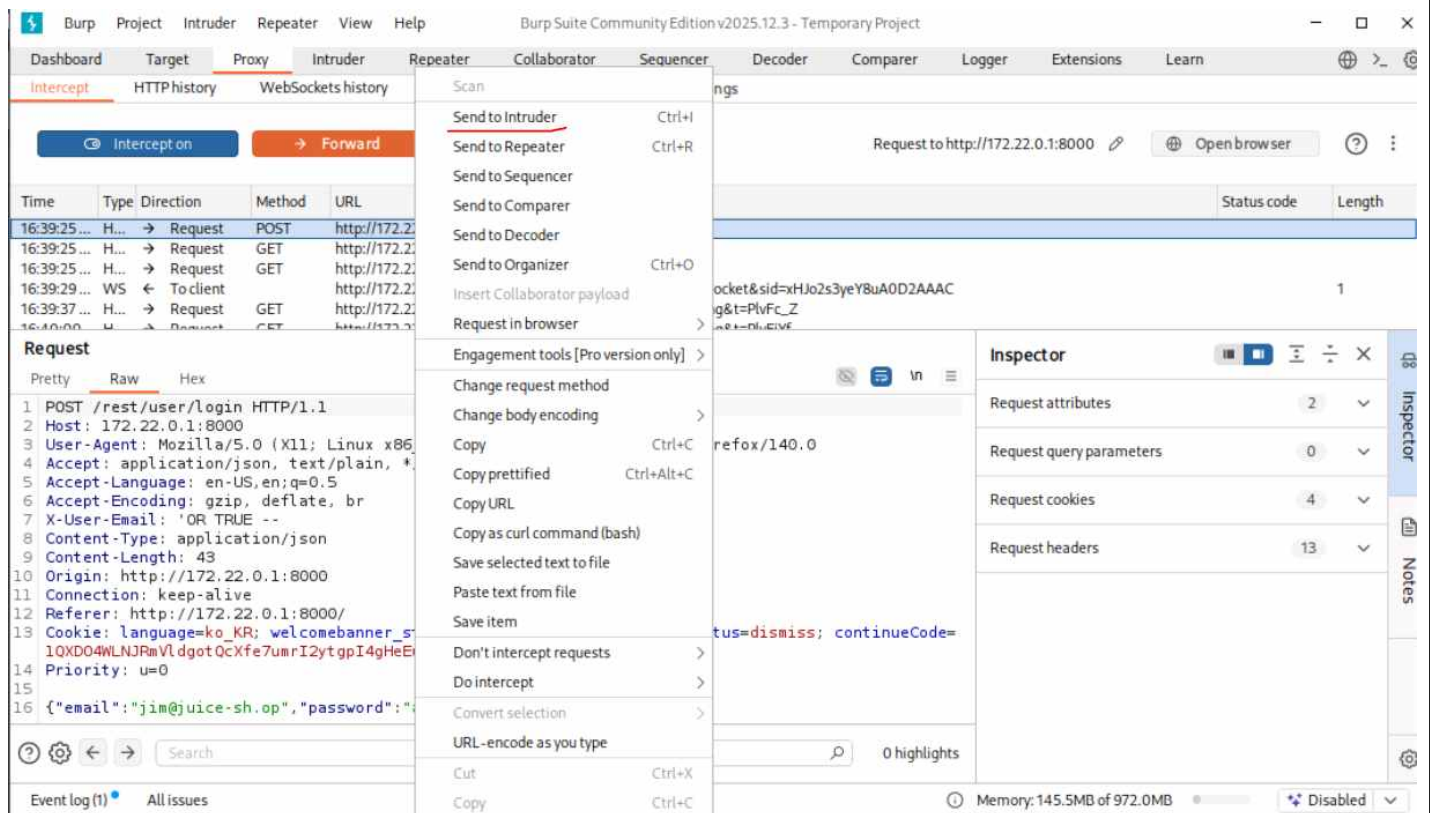
jim@juice-sh.op
bender@juice-sh.op
bjoern.kimminich@gmail.com
ciso@juice-sh.op
support@juice-sh.op
marty@juice-sh.op
mc.safesearch@juice-sh.op
J12934@juice-sh.op
wurstbrot@juice-sh.op

※Brute Force※

1. jim@juice-sh.op의 이메일의 비번에 Brute Force 공격을 할 것임



2. Burp Suit Tool을 사용해서 intercept를 해서 확인함



3. Intruder에 올라서 비번 위치 확인 Add\$를 사용하여 payload 설정한다.

Sniper attack

Target: ☒ Update Host header to match target

Positions:

```
1 POST /rest/user/login HTTP/1.1
2 Host: 172.22.0.1:8000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 X-User-Email: 'OR TRUE --
8 Content-Type: application/json
9 Content-Length: 43
10 Origin: http://172.22.0.1:8000
11 Connection: keep-alive
12 Referer: http://172.22.0.1:8000/
13 Cookie: language=ko_KR; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=1QXD04WLNJRmVLdgotQcXfe7umrI2ytpI4gHeEuorHaXd857PK2MkgBezEY
14 Priority: u=0
15 {"email": "jim@juice-sh.op", "password": "Add$"}
16
```

1 highlight 1 payload position Length: 641

Event log (1) All issues

Memory: 145.5MB of 972.0MB Disabled

Payloads

Payload position: All payload positions

Payload type: Simple list

Payload count: 0

Request count: 0

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Payload processing

4. Cluster bomb attack를 사용하여 정의 된 페이로드를 반복시킨다.

Sniper attack

Target: ☒ Update Host header to match target

Positions:

```
1 POST /rest/user/login HTTP/1.1
2 Host: 172.22.0.1:8000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 X-User-Email: 'OR TRUE --
8 Content-Type: application/json
9 Content-Length: 43
10 Origin: http://172.22.0.1:8000
11 Connection: keep-alive
12 Referer: http://172.22.0.1:8000/
13 Cookie: language=ko_KR; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=1QXD04WLNJRmVLdgotQcXfe7umrI2ytpI4gHeEuorHaXd857PK2MkgBezEY
14 Priority: u=0
15 {"email": "jim@juice-sh.op", "password": "Add$"}
16
```

1 highlight 1 payload position Length: 641

Event log (1) All issues

Memory: 145.5MB of 972.0MB Disabled

Payloads

Payload position: All payload positions

Payload type: Simple list

Payload count: 0

Request count: 0

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Payload processing

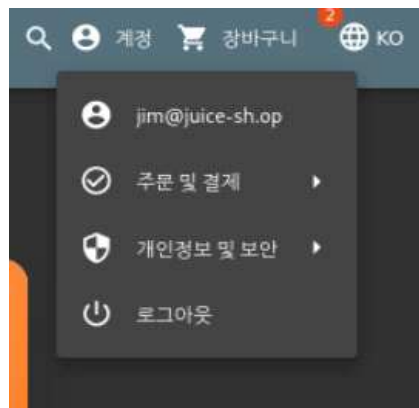
* spider : 각 페이로드에 한 번씩 문자를 넣어준다. ex) payload 1 먼저 다 넣고 다음에 payload 2에 문자를 넣는다.

* clustbomb : 정의된 페이로드를 세트화하여 반복 ex) payload 1 1가지 문자에 payload 2의 문자를 변경을 하면서 넣고 payload2 문자가 끝나면 payload 1의 다른 문자가 대입된다.

5. Git-Hub 상에서 구한 비번 패키지를 그대로 저장하여 비밀번호 무작위 대입을 한다.

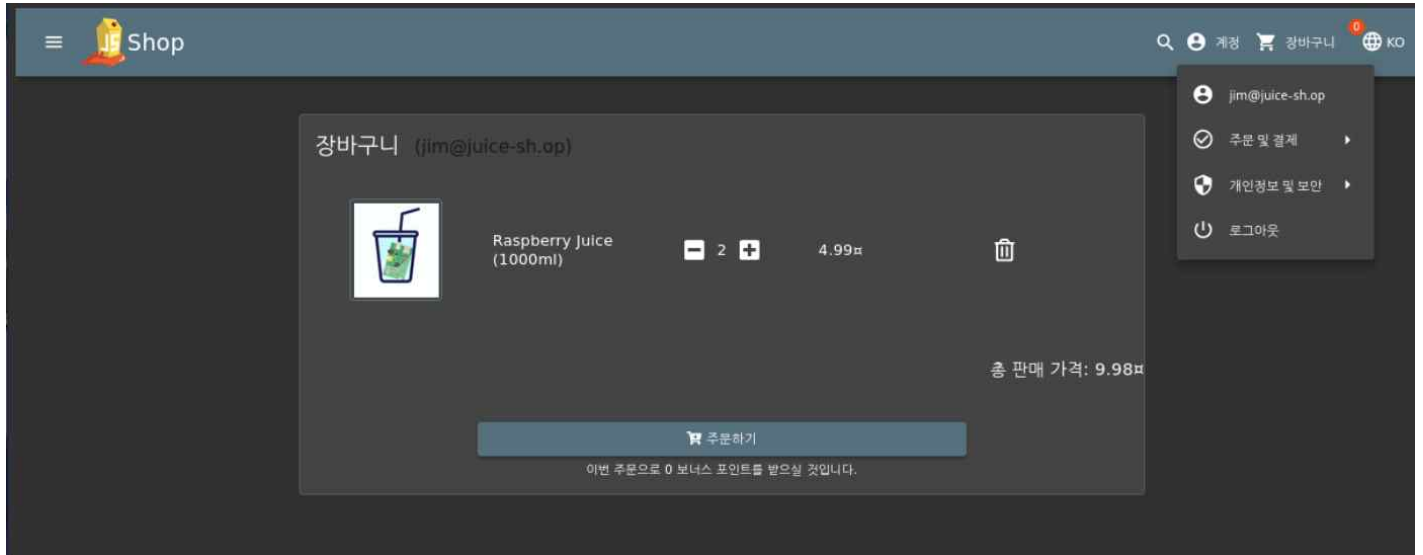
The screenshot shows the Burp Suite Community Edition v2025.12.3 interface. The 'Intruder' tab is active, displaying a 'Cluster bomb attack' configuration. The target is set to 'http://172.22.0.1:8000'. The payload type is 'Brute forcer', and the payload count is 1,679,616. The payload configuration section shows a character set of 'abcdefghijklmnopqrstuvwxyz0123456789' and min/max lengths of 4. The payload processing section is empty. The event log at the bottom shows the attack results, including the target URL and the generated payload: {"email": "jim@juice-sh.op", "password": "\$as\$"}.

6. jim@juice-sh.op의 비번은 jim12345로 확인함

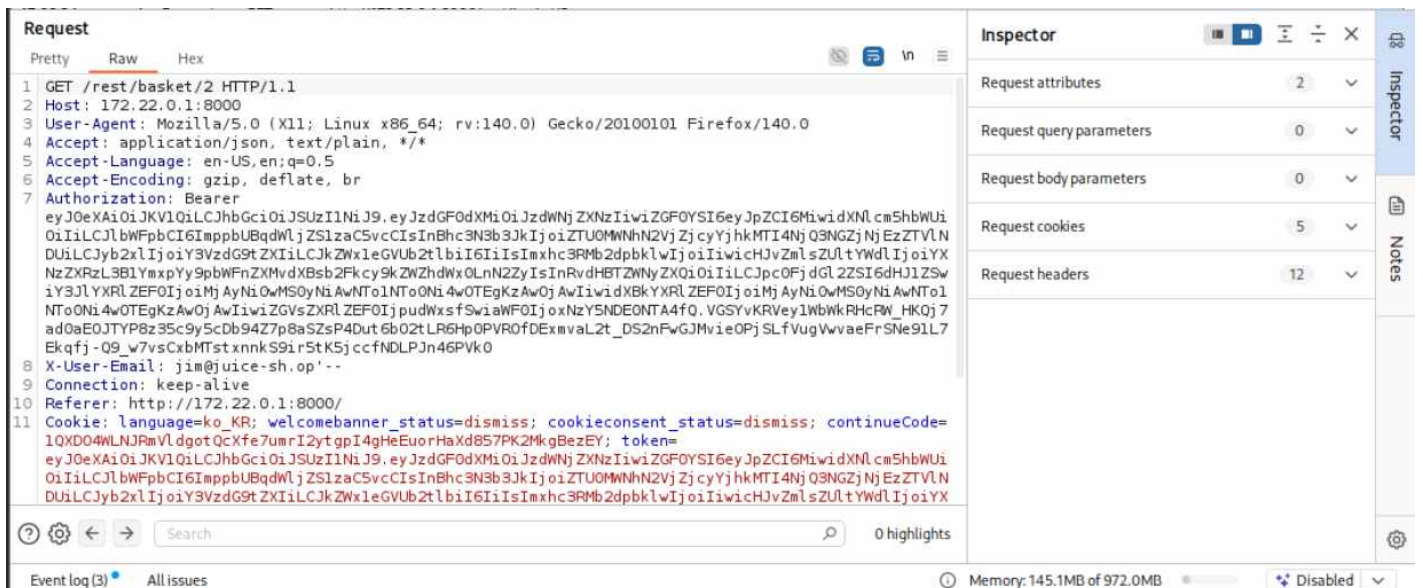


※IDOR (Insecure Direct Object Reference)※

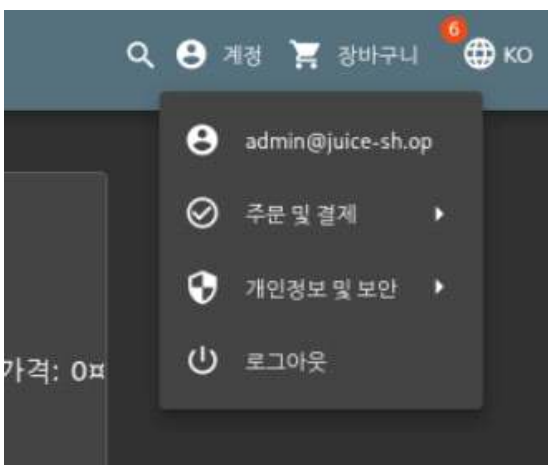
1. jim계정에서 장바구니를 통해 다른 유저의 장바구니를 확인하고 변경시키고 싶다.



2. Burp Suite를 사용하여 Request의 정보를 확인한다.



3. 이미 알아낸 admin 계정 또한 장바구니를 확인한다.



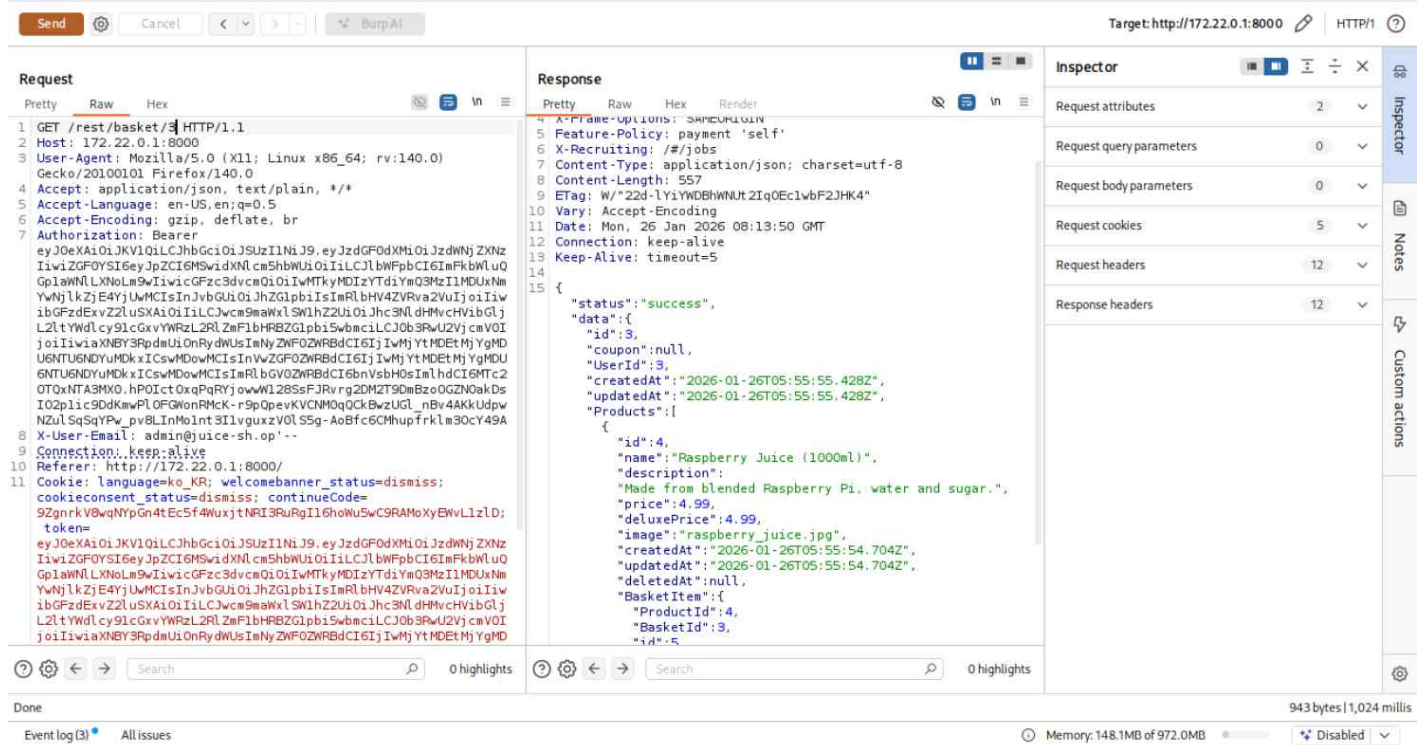
4. Burp Suite를 사용하여 요청 사항의 어느 부분을 유저부분을 구별하는지 확인한다.

[illegible]

5. GET /rest/basket/숫자 HTTP/1.1 즉, 숫자 부분에 따라 구별하는 것을 확인함

[illegible]

6. GET방식을 사용하여 숫자를 변경하여 다른 유저를 확인 할 수 있음을 확인함



※보안 방법※

① IDOR 보안 방법

1. 세션 기반 접근 통제 (필수)

```
SELECT * FROM orders WHERE user id = 요청값;
```

안전한 코드

```
SELECT * FROM orders
WHERE user id = 세션에 저장된 사용자 ID;
```

- 👉 요청 파라미터(userId)를 신뢰하지 말고
- 👉 서버 세션 값으로 조회

2. 서버 측 권한 검증 로직 추가

```
if request.user_id != session.user_id:
    return "403 Forbidden"
```

모든 API에 이 검증 로직이 있어야 함.

3. RBAC 적용 (Role-Based Access Control)

user, admin, manager
역할별 접근 권한 분리
/admin/* → admin만 접근 가능

✓ 4. 예측 불가능한 ID 사용

단순 숫자 ID 대신 UUID 사용
무작위 토큰 사용
/orders/9a8f7b6c-234f-12ab

✓ 5. IDOR 방지 구조

✗ 취약한 코드

```
app.get("/orders/:userId", async (req, res) => {  
  const orders = await Order.find({ userId: req.params.userId });  
  res.json(orders);  
});  
→ URL 값 그대로 사용 → IDOR 발생
```

✓ 안전한 코드

```
app.get("/orders", authenticate, async (req, res) => {  
  const orders = await Order.find({ userId: req.user.id });  
  res.json(orders);  
});
```

※ 보안 방법 ※

- URL에서 userId 받지 않음
- 세션/JWT의 user.id만 사용
- 클라이언트 요청 값 신뢰하지 않음

② Brute Force 방어 방법

✓ 1. 로그인 실패 횟수 제한

5회 실패 → 10분 잠금
일정 횟수 이상 → 관리자 알림

✓ 2. CAPTCHA 적용

자동화 공격 차단

✓ 3. Rate Limiting

1분에 10회 이상 로그인 시도 → 차단

Nginx:

```
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;
```

✓ 4. MFA (다중 인증)

OTP

이메일 인증

2단계 인증

✓ 5. 로그인 로그 모니터링

동일 IP 반복 시도 탐지

비정상 패턴 탐지

③ 관리자 경로 보호 방법

✓ 1. 서버에서 Role 체크

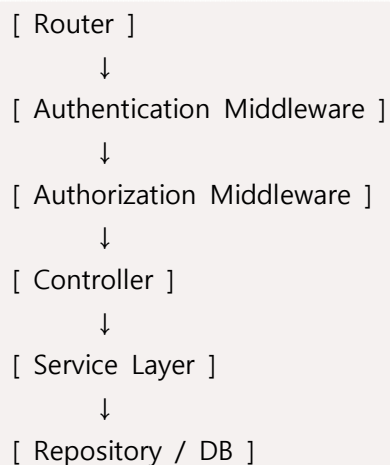
```
if session.role != "admin":  
    return "403 Forbidden"
```

✓ 2. 관리자 API 분리

관리자 전용 서버

별도 인증 체계

✓ 3. 전체 보안 아키텍처 구조



👉 핵심은 Controller에서 직접 DB 접근하지 않는 것

👉 모든 요청은 반드시 인증/인가 레이어를 거치게 설계

✓ 4. 인증(Authentication) 구조

//JavaScript

```
// authController.js
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
async function login(req, res) {
  const { username, password } = req.body;
  const user = await User.findOne({ username });
  if (!user) return res.status(401).send("Invalid credentials");
  const match = await bcrypt.compare(password, user.password);
  if (!match) return res.status(401).send("Invalid credentials");
  const token = jwt.sign(
    { id: user.id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "1h" }
  );
  res.json({ token });
}
```

※ 보안 방법 ※

- 비밀번호는 bcrypt 해시 저장
- JWT에 userId + role 포함
- 세션 대신 토큰 기반 인증 가능

✓ 5. 인증 미들웨어 (모든 요청 보호)

```
// authMiddleware.js
const jwt = require("jsonwebtoken");
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).send("Unauthorized");
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // 여기서 사용자 정보 저장
    next();
  } catch {
    return res.status(401).send("Invalid token");
  }
}
//이제 모든 보호 API는 이 미들웨어를 거침
```